

Práctica en clase. Despliegue de aplicaciones web (Node.js).

Instrucciones. Desplegar una aplicación web sobre Ubuntu Linux sobre VirtualBox.

Es necesario conocer como desplegar una aplicación web en un servidor web para publicarla en producción. Aunque pueda parecer un proceso simple, es una actividad muy delicada y que requiere experiencia para realizarla de manera eficiente y rápida en la práctica.

1. Sigue las instrucciones de cómo instalar Ubuntu desde la práctica [Instalación de Ubuntu server](#).
2. Sigue las instrucciones de cómo instalar Apache desde la práctica [Instalación de Apache Server](#).
3. Crear tu reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdelo con el nombre **NombreAlumno_Despliegue.docx**.
4. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca las capturas de pantalla donde se vea el proceso que seguiste para desplegar tu aplicación web en producción mediante **configuración manual**.
 - b. Coloca las capturas de pantalla donde se vea el proceso que seguiste para desplegar tu aplicación web en producción mediante **contenedores**.
 - c. Coloca las capturas de pantalla donde se vea la aplicación publicada usando SSL (HTTPS).
5. Sube tu reporte de práctica archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito tener instalado el software **Virtual Box**. *Puede seguir las instrucciones de cómo instalar VirtualBox desde la práctica [Instalación de VirtualBox](#).*
2. Esta práctica tiene como prerrequisito contar con un equipo de cómputo con suficientes recursos (Espacio en disco duro, RAM y CPU) para soportar la instalación del software y la ejecución de las máquinas virtuales.
3. Asegúrese de que no tiene ejecutando aplicaciones en los puertos **3000** y **3306** de su servidor. Si es así apague esos servicios o cambie los puertos de la práctica.

Instrucciones: Configuración manual.



1. Siga las instrucciones de cómo instalar Ubuntu desde la práctica [Instalación de Ubuntu server](#). Para Docker es necesario considerar el espacio para las imágenes. Una VM de 40GB es recomendado.
2. Una vez que tenga instalado Ubuntu, ingrese a la consola de su equipo utilizando algún programa cliente de SSH como Putty o Bitvise. No se recomienda trabajar directamente en la ventana de VirtualBox. Piense que en la práctica, usted no tendrá acceso al servidor físico, deberá trabajar de manera remota.
3. Siga las instrucciones de cómo instalar Apache desde la práctica [Instalación de Apache Server](#).
4. Para poder continuar con la práctica, es necesario tener instalado y funcionando su servidor web **Apache**.

Instalación de MySQL

1. Dentro de la consola de MySQL, ejecute los siguiente comandos para instalar MySQL.

```
sudo apt-get update  
sudo apt install mysql-server
```

2. Para revisar el estado del servidor ejecute.

```
sudo service mysql status
```

3. Debe verse su servidor en el estado de **running**.
4. Puede revisar el puerto en que está escuchando MySQL escribiendo.

```
sudo ss -tap | grep mysql
```

5. Para asegurar su instalación con las mejores prácticas de seguridad, normalmente se ejecuta.

```
sudo mysql_secure_installation
```

6. Esto lo llevará a través de una serie de indicaciones donde podrá realizar algunos cambios en las opciones de seguridad de su instalación MySQL. En entornos de producción, se recomienda seleccionar **Sí** a todas las opciones. Como queremos ver cómo queda MySQL de fábrica, presione **No** en todos los casos para que pueda leer lo que el asistente le proporciona.
7. Para iniciar sesión en su servidor, escriba.

```
sudo mysql
```

8. Para revisar los usuarios que pueden acceder a este equipo escriba.

```
SELECT user,authentication_string,plugin,host FROM mysql.user;
```

9. Observe como el plugin para la autenticación del usuario **root** es **auth_socket** y no tiene contraseña.

```
mysql> SELECT user,authentication_string,plugin,host FROM mysql.user;
```

user	authentication_string	plugin	host
debian-sys-maint	\$A\$005\$n;>q5dW`x {9Mv nQQXINedjWgGA0i9U6EU1EZ1ghxqHggbjrTGNVmwCNw4	caching_sha2_password	localhost
mysql.infoschema	\$A\$005\$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED	caching_sha2_password	localhost
mysql.session	\$A\$005\$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED	caching_sha2_password	localhost
mysql.sys	\$A\$005\$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED	caching_sha2_password	localhost
root		auth_socket	localhost

5 rows in set (0,00 sec)

10. Vamos a modificar esto. Primero revise las políticas de seguridad que ya trae su servidor.

```
mysql> SHOW VARIABLES LIKE 'validate_password%';
```

Variable_name	Value
validate_password.changed_characters_percentage	0
validate_password.check_user_name	ON
validate_password.dictionary_file	
validate_password.length	8
validate_password.mixed_case_count	1
validate_password.number_count	1
validate_password.policy	MEDIUM
validate_password.special_char_count	1

8 rows in set (0,01 sec)

11. Necesita una contraseña de al menos 8 caracteres, con 1 numero, 1 mayúscula y 1 carácter especial.

12. Si quisiera cambiar estas políticas, puede hacerlo cambiando los valores; por ejemplo.

```
SET GLOBAL validate_password.length = 6;  
SET GLOBAL validate_password.number_count = 0;
```

13. Pero no lo haga, no es recomendable en entornos de servidores.

14. Ingrese el comando para cambiar la contraseña de **root** siguiendo estas políticas.

```
ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'P@ssw0rd';
```

15. Si vuelve a revisar los usuarios, verá que **root** ya cuenta con una contraseña ahora.

```
mysql> SELECT user,authentication_string,plugin,host FROM mysql.user;
```

user	authentication_string	plugin	host
debian-sys-maint	\$A\$005\$n;>q5dW`x {9Mv nQQXINedjWgGA0i9U6EU1EZ1ghxqHggbjrTGNVmwCNw4	caching_sha2_password	localhost
mysql.infoschema	\$A\$005\$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED	caching_sha2_password	localhost
mysql.session	\$A\$005\$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED	caching_sha2_password	localhost
mysql.sys	\$A\$005\$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED	caching_sha2_password	localhost
root	*8232A1298A49F710DBEE0B330C42EEC825D4190A	mysql_native_password	localhost

5 rows in set (0,00 sec)

16. Salga de la sesión.

```
exit;
```

17. Para volver a ingresar, ahora debe especificar su contraseña y usuario.

```
pato@equipoA:~$ sudo mysql -u root -p  
Enter password:
```

18. En un entorno de producción, generalmente este servidor lo administra un DBA. Usted solo tendría que proporcionar los datos de la creación de la base de datos. Por otro lado, si esta en una arquitectura de microservicios, es probable que usted si tenga que realizar esto en su contenedor.

19. Enhorabuena, continuemos con el despliegue.

Instalación de Git.

1. Para instalar Git en su servidor, solo escriba el siguiente comando.

```
sudo apt-get install git
```

2. Utilizaremos Git como medio de publicación de los archivos de la aplicación.

Instalación de Node.js.

1. En la terminal de Linux, instale el servidor de Node.js.

```
sudo curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

2. Salga de la sesión y vuelva a entrar.

```
nvm install 20  
node -v  
npm -v
```

3. Con esto, su servidor puede alojar contenido desarrollado para esta plataforma.

Copiado de los archivos de la aplicación de Node.js.

1. Si usted quisiera publicar sus propios proyectos, recuerde que tiene varias opciones:

- Puede hacerlo clonando desde GitHub.
- Puede [Compartir su carpeta de trabajo de Windows hacia Linux](#).
- Puede copiar los archivos utilizando herramientas como WinSCP, Bitvise.

2. En esta práctica, lo realizaremos utilizando GitHub.

3. Colóquese en su carpeta `/home/pato/`.

4. Ahora puede clonar el proyecto desde git utilizando.

```
git clone https://github.com/gverafei/mvcnode.git
```

5. Esto creará una nueva carpeta llama `mvcnode` en su directorio personal.

6. Asegúrese de que la carpeta tenga permiso de ejecución. Revise la práctica [Instalación de Apache Server](#) si tiene dudas al respecto.

```
drwxrwxr-x 9 pato pato 4096 may 11 19:59 mvcnode
```

7. Entre a la carpeta `mvnode` y agregue su archivo de secretos.

```
cd mvnode
nano .env
```

8. Ingrese el siguiente contenido.



```
DB_HOST=localhost
DB_DATABASE=clases
DB_USER=clases_user
DB_PASSWORD=P@ssw0rd
SERVER_PORT=3000
NODE_ENV=production
```

9. Enhorabuena, continúe con su despliegue.

Prepare la base de datos de su aplicación.

1. Una vez que cuenta con el script de inicio de su base de datos, entre a la consola de MySQL.

```
pato@equipoA:~$ sudo mysql -u root -p
Enter password:
```

2. Dentro de MySQL, para ejecutar el script de creación de la base de datos, ejecute las siguiente instrucción **source**:

```
mysql> source /home/pato/mvnode/bd.sql
```

3. Verifique que se la base de datos se haya creado.

```
mysql> show databases;
+-----+
| Database |
+-----+
| clases   |
| information_schema |
| mysql    |
| performance_schema |
| sys      |
+-----+
6 rows in set (0.00 sec)
```

4. Ahora vamos a crear el usuario para conectarse desde su aplicación. En este ejemplo le colocaremos el nombre `clases_user` con password `P@ssw0rd`, pero usted puede colocar el que prefiera.

```
mysql> CREATE USER clases_user@localhost IDENTIFIED BY 'P@ssw0rd';
Query OK, 0 rows affected (0.01 sec)
```

5. Y para darle permisos en la nueva base de datos, se utiliza:

```
mysql> GRANT ALL ON clases.* TO clases_user@localhost;
Query OK, 0 rows affected (0.01 sec)
```

6. Para verificar que los permisos se hayan aplicado se ejecuta:

```
mysql> SHOW GRANTS FOR clases_user@localhost;
+-----+
| Grants for clases_user@localhost |
+-----+
| GRANT USAGE ON *.* TO `clases_user`@`localhost` |
| GRANT ALL PRIVILEGES ON `clases`.* TO `clases_user`@`localhost` |
+-----+
2 rows in set (0.00 sec)
```

7. Cerrar la sesión y salir con `exit`.

Instale las dependencias su aplicación de Node.js.

1. Dentro de la carpeta `/home/pato/mvcnode/` instale las dependencias de su proyecto.

```
npm install
```

2. Si es necesario, corrija detalles de seguridad en los paquete.

```
npm audit fix
```

3. Con esto su aplicación esta lista para ser probada.

Pruebe su aplicación de Node.js.

4. Para probar que su aplicación funciona correctamente, dentro de la carpeta `/home/pato/mvcnode`, inicie su servidor de node.

```
pato@equipoA:~/mvcnode$ node index.js
Aplicación de ejemplo escuchando en el puerto 3000
```

5. Abra un navegador web en su equipo host y revise que en el puerto 3000 puede ver su aplicación ejecutándose de manera correcta. Por ejemplo `http://192.168.56.102:3000/`.



Alumnos con materias

Lista de alumnos

Id: 1 Sofía Martínez López Materia: Tecnologías web Promedio: 9.2 Detalles	Id: 2 Miguel Ángel Torres Ruiz Materia: Estructuras de datos Promedio: 8.5 Detalles	Id: 3 Carolina Herrera Gutiérrez Materia: Bases de datos Promedio: 9.8 Detalles	Id: 4 Laura González Materia: Integración Promedio: 8.0 Detalles
Id: 5	Id: 6	Id: 7	Id: 8

6. Enhorabuena, su aplicación funciona en producción.

Utilizar un gestor de procesos de Node.js.

PM2 es un gestor de procesos para aplicaciones en producción de Node. Es capaz de soportar cantidades enormes de tráfico con un consumo de recursos optimizado para grandes volúmenes y con herramientas que facilitan la administración de tus Apps.

Entre otras cosas PM2 le permitirá:

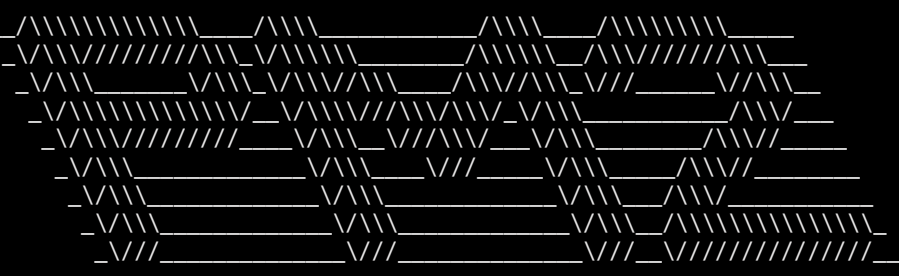
- Iniciar, reiniciar, recargar, detener o eliminar una App de Node.js, Python, Golang, etc.
- Detectar y aplicar cambios de tu App automáticamente.
- Monitorear y diagnosticar tu App vía terminal o interfaz web (verificar estado, consumos, logs, métricas, etc.).
- Reiniciar automáticamente tu App si se bloquea o se detiene por un error inesperado.
- Ejecutar tu App en modo alta disponibilidad (HA) o clúster (con balanceo de carga entre procesos).
- Administrar múltiples aplicaciones multi-plataforma en una misma interfaz.

1. Para instalar PM2 ejecute.

```
npm install pm2 -g
```

2. Una vez instalado, ahora puede agregar su aplicación al gestor y cada vez que el servidor inicie, su aplicación también lo hará.

```
pato@equipoA:~/mvcnode$ pm2 start index.js
```



```
Runtime Edition

PM2 is a Production Process Manager for Node.js applications
with a built-in Load Balancer.
```

3. Observe que ahora su aplicación se ejecuta de una manera profesional.

```
[PM2] Spawning PM2 daemon with pm2_home=/home/pato/.pm2
[PM2] PM2 Successfully daemonized
[PM2] Starting /home/pato/mvcnode/index.js in fork_mode (1 instance)
[PM2] Done.
```

id	name	namespace	version	mode	pid	uptime	🔄	status	cpu	mem	user	watching
0	index	default	1.0.0	fork	3373	0s	0	online	0%	30.9mb	pato	disabled

```
pm2 list
```

```
pm2 monit
```

6. Para detener su aplicación ejecute.

```
pm2 stop index.js
```

```
pm2 delete index.js
```

8. Si desea que su aplicación se ejecute en cada reinicio ejecute `pm2 startup`.

```
pato@equipoa:~$ pm2 startup
[PM2] Init System found: systemd
[PM2] To setup the Startup Script, copy/paste the following command:
sudo env PATH=$PATH:/home/pato/.npm/versions/node/v20.13.1/bin
/home/pato/.npm/versions/node/v20.13.1/lib/node_modules/pm2/bin/pm2 startup systemd -u pato --hp
/home/pato
```

9. Copie todo el comando que pm2 le proporciona (marcado en amarillo) y ejecútelo en otra línea.

```
sudo env PATH=$PATH:/home/pato/.nvm/versions/node/v20.13.1/bin
/home/pato/.nvm/versions/node/v20.13.1/lib/node_modules/pm2/bin/pm2 startup systemd -u pato --hp
/home/pato
```

10. Finalmente ejecute el siguiente comando para guardar la lista de aplicaciones.

```
pm2 save
```

11. Ahora puede reiniciar su servidor con `sudo reboot now` y su aplicación se iniciará de manera automática.
12. Enhorabuena, su aplicación funciona en producción.

Publicar su aplicación en un servidor Web.

1. Una vez que su aplicación está funcionando de manera consistente, el siguiente paso es exponerla hacia el mundo mediante un servidor web.
2. Instale los módulos para realizar un proxy inverso con los comandos.

```
sudo a2enmod proxy
sudo a2enmod proxy_http
sudo service apache2 restart
```

3. Abra su archivo de configuración de **HTTPS** que tiene SSL desde `sudo nano /etc/apache2/sites-available/000-default-ssl.conf` y coloque el curso debajo del `DocumentRoot`.
4. Agregue lo marcado en amarillo. Su archivo debería quedar de esta manera.

```
<VirtualHost *:443>
    ServerName fei.uv.mx
    DocumentRoot /var/www/html

    ProxyRequests Off
    ProxyPreserveHost On
    <Location /mvcnode>
        ProxyPass http://127.0.0.1:3000
        ProxyPassReverse http://127.0.0.1:3000
    </Location>

    <IfModule alias_module>
        Alias /camara /home/pato/camara
        <Directory /home/pato/camara>
            Options FollowSymLinks
            AllowOverride None
            Require all granted
        </Directory>
    </IfModule>

    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/apache-selfsigned.crt
    SSLCertificateKeyFile /etc/ssl/private/apache-selfsigned.key
</VirtualHost>
```

5. Para guardar los cambios, presione `Ctrl + O` y `Enter`. Para salir `Ctrl + X`.
6. Para revisar que su configuración no marque algún error ejecute `sudo apache2ctl configtest`. Y revise que no aparezca ningún mensaje de error, salvo la advertencia de que no tenemos un dominio.

```
pato@equipoA:~$ sudo apache2ctl configtest
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 10.0
2.15. Set the 'ServerName' directive globally to suppress this message
Syntax OK
```

7. En caso de error, le dirá el motivo y la línea a corregir de su archivo.
8. Haga lo mismo con el archivo de configuración de **HTTP** (sin SSL) desde `sudo nano /etc/apache2/sites-available/000-default.conf` y coloque el curso debajo del `DocumentRoot`.

```
<VirtualHost *:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
```

```
# specifies what hostname must appear in the request's Host: header to
# match this virtual host. For the default virtual host (this file) this
# value is not decisive as it is used as a last resort host regardless.
# However, you must set it for any further virtual host explicitly.
#ServerName www.example.com

ServerName fei.uv.mx
ServerAdmin webmaster@localhost
DocumentRoot /var/www/html

ProxyRequests Off
ProxyPreserveHost On
<Location /mvcnode>
    ProxyPass http://127.0.0.1:3000
    ProxyPassReverse http://127.0.0.1:3000
</Location>

<IfModule alias_module>
    Alias /camara /home/pato/camara
    <Directory /home/pato/camara>
        Options FollowSymLinks
        AllowOverride None
        Require all granted
    </Directory>
</IfModule>
```

9. Si la sintaxis marca OK, puede reiniciar su servidor Apache para aplicar los cambios.

```
sudo service apache2 restart
```

10. Ahora prueba su aplicación desde un navegador web utilizando el puerto **80** o el **443**. Por ejemplo **http://192.168.56.102/mvcnode/**.

Alumnos con materias

Lista de alumnos

Id: 1	Id: 2	Id: 3	Id: 4
Sofía Martínez López	Miguel Ángel Torres Ruiz	Carolina Herrera Gutiérrez	Laura González Sánchez
Materia: Tecnologías web	Materia: Estructuras de datos	Materia: Bases de datos	Materia: Integración de soluciones
Promedio: 9.2	Promedio: 8.5	Promedio: 9.8	Promedio: 8
Detalles	Detalles	Detalles	Detalles

11. Ahora puede publicar múltiples aplicaciones, en diferentes directorios y en el mismo puerto sobre Apache.
12. Intente realizar la publicación de otra aplicación en su servidor web de producción.
13. Enhorabuena, su aplicación esta lista para producción.

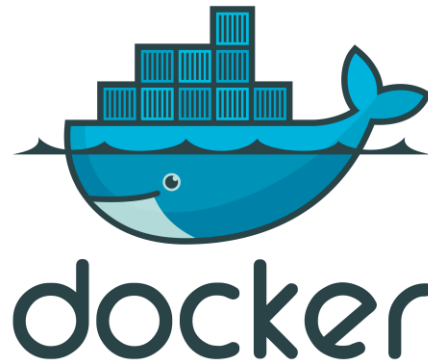
Habilitar el firewall.

1. Para terminar de asegurar su servidor web de producción, habilite el firewall solo para el servidor web Apache. Sus clientes no necesitan conocer otros puertos más que los de **SSH** y **HTTP**.
2. Tenga mucho cuidado en el comando **sudo ufw enable**. Asegúrese de que agregó la regla para habilitar SSH porque podría dejarlo fuera de la sesión.

```
sudo ufw status
sudo ufw app list
sudo ufw allow OpenSSH
sudo ufw allow 'Apache Full'
sudo ufw enable
sudo ufw status verbose
```

3. Esto realmente depende de sus necesidades. Hay tres perfiles disponibles para Apache:
 - a. **Apache**: este perfil abre solo el puerto 80 (tráfico web normal sin cifrar)
 - b. **Apache Full**: este perfil abre tanto el puerto 80 (tráfico web normal sin cifrar) como el puerto 443 (tráfico cifrado TLS/SSL)
 - c. **Apache Secure**: este perfil abre solo el puerto 443 (tráfico cifrado TLS/SSL)
4. Se recomienda que habilite el perfil más restrictivo que aún permitirá el tráfico que ha configurado.
5. Para deshabilitar el firewall, ingrese **sudo ufw disable**.
6. Enhorabuena, ha realizado el despliegue de manera correcta.

Instrucciones: Mediante contenedores.



1. Siga las instrucciones de cómo instalar Ubuntu desde la práctica [Instalación de Ubuntu server](#).
2. Una vez que tenga instalado Ubuntu, ingrese a la consola de su equipo utilizando algún programa cliente de SSH como Putty o Bitvise. No se recomienda trabajar directamente en la ventana de VirtualBox. Piense que en la práctica, usted no tendrá acceso al servidor físico, deberá trabajar de manera remota.
3. Siga las instrucciones de cómo instalar Apache desde la práctica [Instalación de Apache Server](#).
4. Si realizó la sección anterior, asegúrese de que no tiene ejecutando la app en PM2 y en MySQL para que las siguientes acciones no interfieran en el uso de los puertos 3000, 3306.

```
sudo service mysql stop  
pm2 delete 0
```

5. Claro que puede colocar sus contenedores en distintos puertos también. Pero para fines prácticos, todo quedará en los mismos puertos que la sección anterior.

Instalación de Docker

1. Primero se deben agregar el repositorio de Docker a su equipo. Asegurese de contar con suficiente espacio en disco duro para las imágenes. Copie y pegue las siguientes instrucciones para ello.

```
# Add Docker's official GPG key:  
sudo apt-get update  
sudo apt-get install ca-certificates curl  
sudo install -m 0755 -d /etc/apt/keyrings  
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc  
sudo chmod a+r /etc/apt/keyrings/docker.asc  
  
# Add the repository to Apt sources:  
echo \\\n  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \\\n  https://download.docker.com/linux/ubuntu \\\n  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \\\n  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null  
sudo apt-get update
```

2. Ahora ya puede instalar los paquetes de Docker.

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

3. Para revisar que su instalación esta correcta, ejecute.

```
sudo docker version
```

4. Para ejecutar Docker sin estar escribiendo la palabra `sudo`, ejecute.

```
sudo groupadd docker  
sudo usermod -aG docker $USER
```

5. Necesitará cerrar la sesión para que los cambios se apliquen.
6. Enhorabuena, continúe con su deployment.

Instalación de Git.

1. Para instalar Git en su servidor, solo escriba el siguiente comando.

```
sudo apt-get install git
```

2. Utilizaremos Git como medio de publicación de los archivos de la aplicación.

Copiado de los archivos de la aplicación de Node.js.

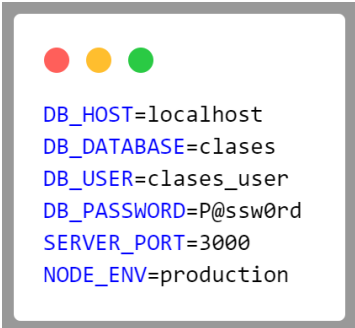
1. Si usted quisiera publicar sus propios proyectos, recuerde que tiene varias opciones:
 - a. Puede hacerlo clonando desde GitHub.
 - b. Puede [Compartir su carpeta de trabajo de Windows hacia Linux](#).
 - c. Puede copiar los archivos utilizando herramientas como WinSCP, Bitvise.
2. En esta práctica, lo realizaremos utilizando GitHub.
3. Colóquese en su carpeta `/home/pato/`.
4. Ahora puede clonar el proyecto desde git utilizando. Si ya lo hizo, no lo vuelva a realizar.

```
git clone https://github.com/gverafei/mvcnode.git
```

5. Esto creará una nueva carpeta llama `mvcnode` en su directorio personal.
6. Entre a la carpeta `mvcnode` y agregue su archivo de secretos.

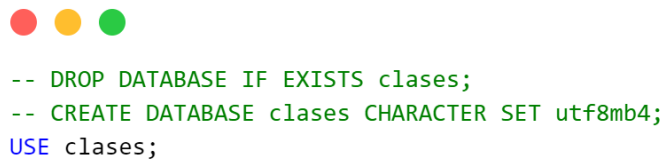
```
cd mvcnode  
nano .env
```

10. Ingrese el siguiente contenido.



```
DB_HOST=localhost  
DB_DATABASE=clases  
DB_USER=clases_user  
DB_PASSWORD=P@ssw0rd  
SERVER_PORT=3000  
NODE_ENV=production
```

7. Ingrese al archivo `bd.sql` y elimine o comente las dos primeras líneas de la creación de la base de datos pues ya no son necesarias.



```
-- DROP DATABASE IF EXISTS clases;  
-- CREATE DATABASE clases CHARACTER SET utf8mb4;  
USE clases;
```

8. Enhorabuena, continúe con su despliegue.

Instalación de MySQL

1. Obtenga la imagen oficial de MySQL.

```
docker pull mysql:latest
```

2. Verifique que la imagen se encuentra ya en su sistema local.

```
pato@equipoA:~$ docker images  
REPOSITORY TAG IMAGE ID CREATED SIZE  
mysql latest e9387c13ed83 10 days ago 578MB
```

3. Si quisiera remover esta imagen. No lo haga, pues tendrá que volver a descargarla.

```
docker rmi mysql -f
```

4. En su carpeta `/home/patito/`, ejecute el siguiente comando para eliminar volúmenes creados previamente.

```
docker volume rm fei-mysql-data  
sudo rm -r fei-mysql-data
```

5. Para crear una nuevo contenedor de MySQL con almacenamiento persistente y crea nuestra base de datos y usuario en el puerto 3306.

```
docker run --name fei-mysql -e MYSQL_ROOT_PASSWORD=P@ssw0rd -p 3306:3306 -v ~/fei-mysql-data:/var/lib/mysql -v ~/mvcnode:/docker-entrypoint-initdb.d -e MYSQL_USER=clases_user -e MYSQL_PASSWORD=P@ssw0rd -e MYSQL_DATABASE=clases --restart always -d mysql
```

6. Para revisar sus contenedores en ejecución.

```
docker ps  
docker ps -al
```

7. Puede revisar el log de MySQL. Quédese viendo la salida hasta que su servidor termine de iniciar.

```
docker logs -f fei-mysql
```

8. Puede revisar el puerto de MySQL.

```
pato@equipoA:~$ docker port fei-mysql
3306/tcp -> 0.0.0.0:3306
3306/tcp -> [::]:3306
```

9. Para detener su contenedor.

```
docker stop fei-mysql
```

10. Para iniciar su contenedor.

```
docker start fei-mysql
```

11. Para removerlo aun cuando este en ejecución. Tendrá que volverlo a crear.

```
docker rm fei-mysql -f
```

12. Puede conectarse a la consola de MySQL utilizando. Observe que utilizamos `--host=127.0.0.1`.

```
sudo apt install mysql-client
mysql --host=127.0.0.1 --port=3306 -u root -p
```

13. Para revisar los usuarios que pueden acceder a este equipo escriba.

```
SELECT user,authentication_string,plugin,host FROM mysql.user;
```

14. Observe como los usuarios son ligeramente distintos a la instalación manual. Ya está el usuario de la aplicación agregado con los privilegios a la base de datos requerida.

```
mysql> SELECT user,authentication_string,plugin,host FROM mysql.user;^M
+-----+-----+-----+-----+
| user          | authentication_string | plugin          | host |
+-----+-----+-----+-----+
| clases_user   | $A$005$_BxE:{^`n\?Cm*!7:Q1zHhZZdKz2AW46RKu/i73veIKHfXGV5rUImtidsjXV3 | caching_sha2_password | %    |
| root          | $A$005$jP_y"o#9{wM]  ?%8< S;jy89iPmHF102H1SZSaISEkaEj5n88jRf2goZa1o/rL6 | caching_sha2_password | %    |
| mysql.infoschema | $A$005$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED | caching_sha2_password | localhost |
| mysql.session | $A$005$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED | caching_sha2_password | localhost |
| mysql.sys     | $A$005$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED | caching_sha2_password | localhost |
| root          | $A$005$j(NRMG\%03@b! (~Iwk1H8vcxHehWw6jAejBqEWKz1.E2t574R2q15cIdSQc | caching_sha2_password | localhost |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

15. Observe como no hay políticas de seguridad en esta imagen.

```
mysql> SHOW VARIABLES LIKE 'validate_password%';
Empty set (0.00 sec)
```

16. Verifique que se la base de datos se haya creado.

```
mysql> show databases;
+-----+
| Database |
+-----+
| clases   |
| information_schema |
| mysql    |
| performance_schema |
| sys      |
+-----+
6 rows in set (0.00 sec)
```

17. Para verificar que los permisos se hayan aplicado se ejecuta:

```
mysql> SHOW GRANTS FOR clases_user;
+-----+
| Grants for clases_user@localhost |
+-----+
| GRANT USAGE ON *.* TO `clases_user`@`localhost` |
| GRANT ALL PRIVILEGES ON `clases`.* TO `clases_user`@`localhost` |
+-----+
2 rows in set (0.00 sec)
```

18. Cerrar la sesión y salir con `exit`.

19. En un entorno de producción, generalmente este servidor lo administra un DBA. Usted solo tendría que proporcionar los datos de la creación de la base de datos. Por otro lado, si está en una arquitectura de microservicios, es probable que usted si tenga que realizar esto en su contenedor.

20. Enhorabuena, continuemos con el despliegue.

Prepare la base de datos de su aplicación.

1. Ya no hay necesidad de preparar la base de datos, pues ya está todo dentro del contenedor de MySQL creado anteriormente.

Instalación de Node.js.

1. Ingrese su carpeta del proyecto `/home/pato/mvcnode`.
2. Revise que exista el archivo llamado `Dockerfile`.

```
nano Dockerfile
```

3. Vea el contenido del archivo.

```
# Usa Node 20 alpine como la imagen padre
FROM node:20-alpine

# Se cambia al directorio /app dentro de la imagen
WORKDIR /app

# Copia los archivos package.json y package-lock.json al directorio /app de la imagen
COPY package.json package-lock.json ./

# Instala las dependencias
RUN npm install

# Copia todos los archivos a la imagen
COPY . .

# Pone la app en el puerto
EXPOSE 3000

# Inicia la aplicacion
CMD npm start
```

4. Ahora desde este mismo directorio donde está el `Dockerfile`, construya su nueva imagen.

```
docker build . -t imgmvcnode
```

5. Revise sus imágenes disponibles.

```
pato@equipoA:~/mvcnode$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
imgmvcnode    latest    2deba4702239   3 minutes ago  148MB
mysql         latest    e9387c13ed83   10 days ago    578MB
```

6. Ahora puede crear un contenedor basado en esta imagen.

```
docker run --name mvcnode --network="host" --restart always -d imgmvcnode
```

21. Para revisar sus contenedores en ejecución.

```
docker ps
```

22. Puede revisar el log del contenedor. Quédese viendo la salida hasta que su servidor termine de iniciar.

```
docker logs -f mvcnode
```

23. Puede revisar el puerto de MySQL.

```
pato@equipoA:~$ docker port mvcnode
3000/tcp -> 0.0.0.0:3000
3000/tcp -> [::]:3000
```

24. Para conectarse a la consola del contenedor.

```
docker exec -it mvcnode sh
```

25. Para detener su contenedor.

```
docker stop mvcnode
```

26. Para iniciar su contenedor.

```
docker start mvcnode
```

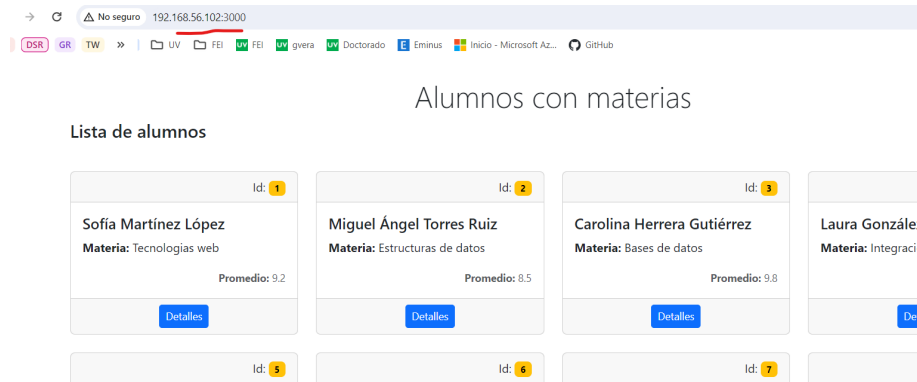
27. Para removerlo aun cuando este en ejecución. Tendrá que volverlo a crear.

```
docker rm mvcnode -f
```

28. Con esto, su servidor ya está alojando la aplicación en el puerto seleccionado.

Pruebe su aplicación de Node.js.

1. Abra un navegador web en su equipo host y revise que en el puerto 3000 puede ver su aplicación ejecutándose de manera correcta. Por ejemplo `http://192.168.56.102:3000/`.



2. Enhorabuena, su aplicación funciona en producción.

Publicar su aplicación en un servidor Web.

1. Una vez que su aplicación está funcionando de manera consistente, el siguiente paso es exponerla hacia el mundo mediante un servidor web.
2. Instale los módulos para realizar un proxy inverso con los comandos.

```
sudo a2enmod proxy
sudo a2enmod proxy_http
sudo service apache2 restart
```

3. Abra su archivo de configuración de **HTTPS** que tiene SSL desde `sudo nano /etc/apache2/sites-available/000-default-ssl.conf` y coloque el curso debajo del **DocumentRoot**.
4. Agregue lo marcado en amarillo. Su archivo debería quedar de esta manera.

```
<VirtualHost *:443>
    ServerName fei.uv.mx
    DocumentRoot /var/www/html

    ProxyRequests Off
    ProxyPreserveHost On
    <Location /mvcnode>
        ProxyPass http://127.0.0.1:3000
        ProxyPassReverse http://127.0.0.1:3000
    </Location>

    <IfModule alias_module>
        Alias /camara /home/pato/camara
        <Directory /home/pato/camara>
            Options FollowSymLinks
            AllowOverride None
            Require all granted
        </Directory>
    </IfModule>

    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/apache-selfsigned.crt
```

```
SSLCertificateKeyFile /etc/ssl/private/apache-selfsigned.key
</VirtualHost>
```

5. Para guardar los cambios, presione **Ctrl + O** y **Enter**. Para salir **Ctrl + X**.
6. Para revisar que su configuración no marque algún error ejecute **sudo apache2ctl configtest**. Y revise que no aparezca ningún mensaje de error, salvo la advertencia de que no tenemos un dominio.

```
pato@equipoA:~$ sudo apache2ctl configtest
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 10.0
2.15. Set the 'ServerName' directive globally to suppress this message
Syntax OK
```

7. En caso de error, le dirá el motivo y la línea a corregir de su archivo.
8. Haga lo mismo con el archivo de configuración de **HTTP** (sin SSL) desde **sudo nano** **/etc/apache2/sites-available/000-default.conf** y coloque el curso debajo del **DocumentRoot**.

```
<VirtualHost *:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerName fei.uv.mx
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html

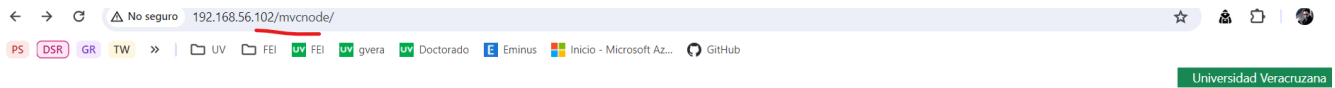
    ProxyRequests Off
    ProxyPreserveHost On
    <Location /mvcnode>
        ProxyPass http://127.0.0.1:3000
        ProxyPassReverse http://127.0.0.1:3000
    </Location>

    <IfModule alias_module>
        Alias /camara /home/pato/camara
        <Directory /home/pato/camara>
            Options FollowSymlinks
            AllowOverride None
            Require all granted
        </Directory>
    </IfModule>
```

9. Si la sintaxis marca OK, puede reiniciar su servidor Apache para aplicar los cambios.

```
sudo service apache2 restart
```

10. Ahora prueba su aplicación desde un navegador web utilizando el puerto **80** o el **443**. Por ejemplo **http://192.168.56.102/mvcnode/**.



Id: 1	Id: 2	Id: 3	Id: 4
Sofía Martínez López	Miguel Ángel Torres Ruiz	Carolina Herrera Gutiérrez	Laura González Sánchez
Materia: Tecnologías web	Materia: Estructuras de datos	Materia: Bases de datos	Materia: Integración de soluciones
Promedio: 9.2	Promedio: 8.5	Promedio: 9.8	Promedio: 8
Detalles	Detalles	Detalles	Detalles

11. Ahora puede publicar múltiples aplicaciones, en diferentes directorios y en el mismo puerto sobre Apache.
12. Intente realizar la publicación de otra aplicación en su servidor web de producción.
13. Enhorabuena, su aplicación esta lista para producción.

Habilitar el firewall.

1. Para terminar de asegurar su servidor web de producción, habilite el firewall solo para el servidor web Apache. Sus clientes no necesitan conocer otros puertos más que los de **SSH** y **HTTP**.
2. Tenga mucho cuidado en el comando **sudo ufw enable**. Asegúrese de que agregó la regla para habilitar SSH porque podría dejarlo fuera de la sesión.

```
sudo ufw status
sudo ufw app list
sudo ufw allow OpenSSH
sudo ufw allow 'Apache Full'
sudo ufw enable
sudo ufw status verbose
```

3. Esto realmente depende de sus necesidades. Hay tres perfiles disponibles para Apache:
 - a. **Apache**: este perfil abre solo el puerto 80 (tráfico web normal sin cifrar)
 - b. **Apache Full**: este perfil abre tanto el puerto 80 (tráfico web normal sin cifrar) como el puerto 443 (tráfico cifrado TLS/SSL)
 - c. **Apache Secure**: este perfil abre solo el puerto 443 (tráfico cifrado TLS/SSL)
4. Se recomienda que habilite el perfil más restrictivo que aún permitirá el tráfico que ha configurado.
5. Para deshabilitar el firewall, ingrese **sudo ufw disable**.
6. Enhorabuena, ha realizado el despliegue de manera correcta.